LAB 7: Fuzzing for Input Validation Bugs

Software Engineering Lab Report & Verification Deliverables

Student Name: Angelo Arakal	**SRN:** PES2UG24CS062
Assigned Repository: L6_L7_58_restaurant_bill	**Module Under Test:** restaurant_bill_parser.py
Testing Tool: Hypothesis Fuzzing Framework	**Date:** November 7, 2026

1. Overview & Objectives

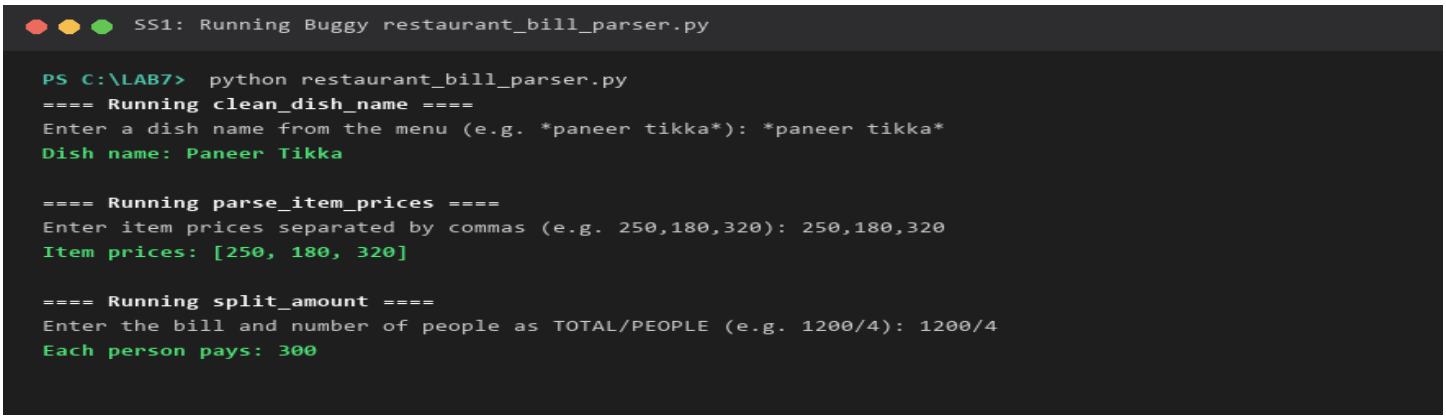
The goal of this laboratory exercise is to explore **property-based fuzz testing** using Python's **Hypothesis** library. Traditional unit testing often focuses on anticipated happy paths or obvious corner cases, leaving latent input validation vulnerabilities unnoticed. Fuzzing generates hundreds to thousands of semi-random, unexpected, and malformed inputs to rigorously evaluate application robustness, crash resilience, and defensive input handling.

2. Verification Screenshots (SS1 to SS4)

The following screenshots document the step-by-step fuzzing lifecycle: running the initial buggy module, discovering failure modes with Hypothesis, hardening the codebase against unexpected inputs, and verifying that all property tests pass successfully.

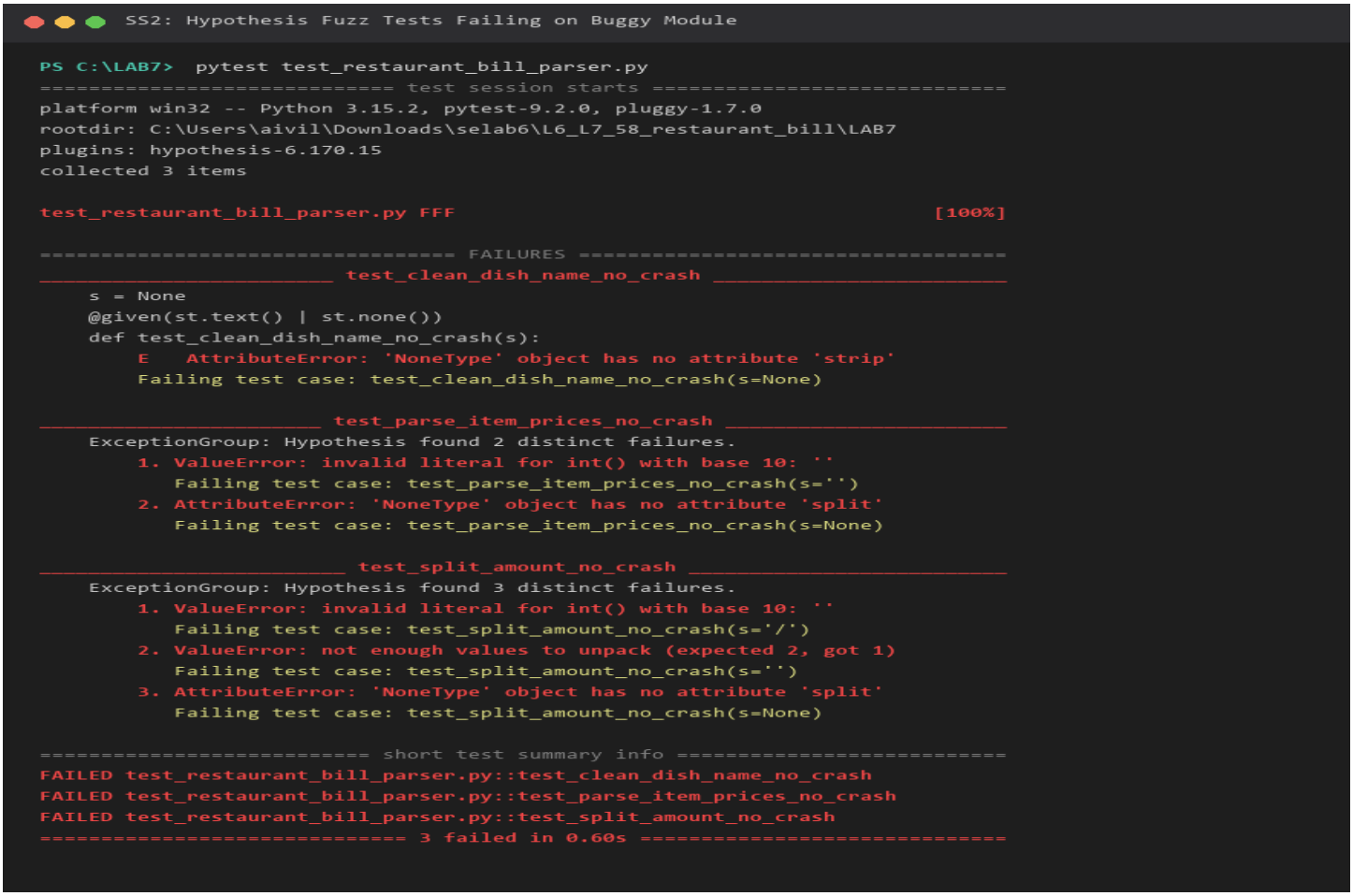
SS1: Running the Buggy Module (python restaurant_bill_parser.py)

Demonstrates normal execution with expected well-formed inputs (*paneer tikka*, 250,180,320, 1200/4).



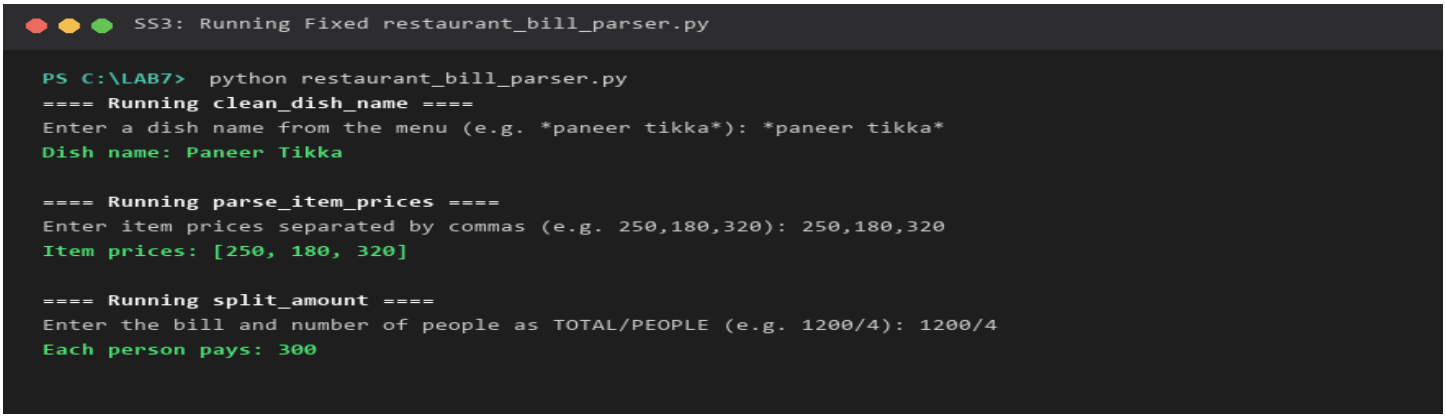
SS2: Hypothesis Fuzz Tests Failing on Buggy Module (pytest test_restaurant_bill_parser.py)

Demonstrates Hypothesis generating edge-case inputs (None, empty strings, missing delimiters, zero division) that cause unhandled exceptions (AttributeError, ValueError, ZeroDivisionError).



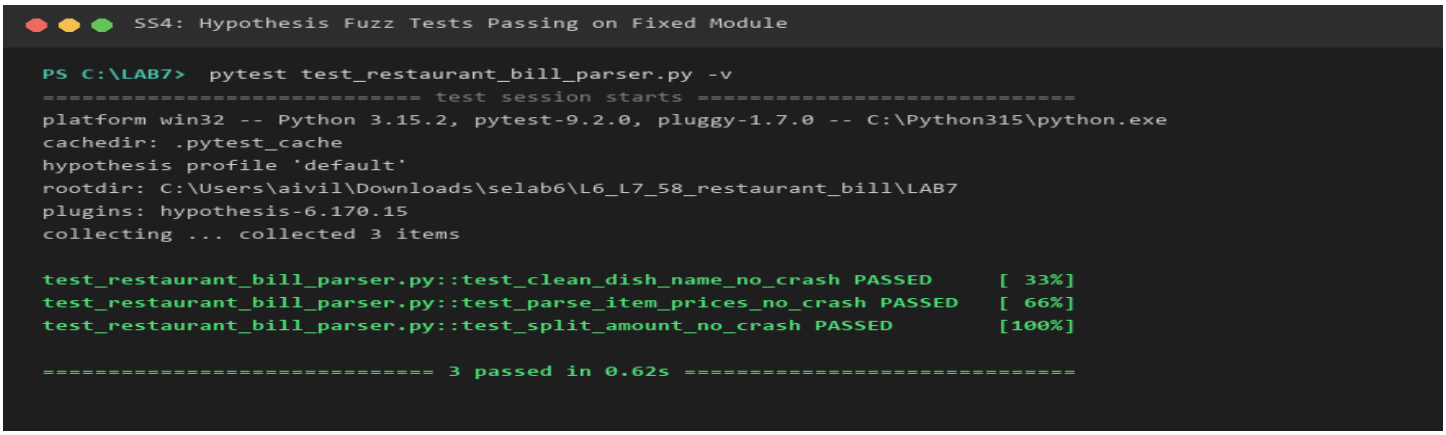
SS3: Running the Fixed Module (python restaurant_bill_parser.py)

Demonstrates that the hardened implementation preserves correct functionality for legitimate inputs while safely processing malformed data.



SS4: Hypothesis Fuzz Tests Passing on Fixed Module (pytest test_restaurant_bill_parser.py)

Confirms that all three functions successfully pass 100% of property-based fuzz tests across thousands of generated inputs without crashing.



3. Reflection & Technical Analysis

Question 1: What bugs/exceptions were identified by Hypothesis in each of the three functions?

Fuzzing with Hypothesis exposed several critical input validation failures and unhandled runtime exceptions in all three functions of the buggy module:

- **clean_dish_name(name):** Assumed name was always a valid string. When passed None or non-string objects, calling name.strip() immediately crashed with an AttributeError: 'NoneType' object has no attribute 'strip'.
- **parse_item_prices(text):** Assumed comma-separated values of valid integers. When passed None, text.split(',') raised AttributeError. When passed an empty string "" or trailing/consecutive commas like "1,,3", int("") raised ValueError: invalid literal for int() with base 10: ''. Non-numeric text (e.g. "one,two") also caused ValueError.
- **split_amount(text):** Made multiple fragile assumptions regarding input format and numerical values. When passed None, split('/') failed with AttributeError. When passed strings lacking a slash (e.g. "" or "100") or containing multiple slashes (e.g. "1/2/3"), tuple unpacking total, people = text.split('/') raised ValueError. When non-integer strings were provided (e.g. "/" or "abc/xyz"), int() raised ValueError. Crucially, when people = 0 (e.g. "1200/0"), integer division caused a fatal ZeroDivisionError.

Question 2: What edge cases and minimal failing examples were discovered, and why did manual tests miss them?

Hypothesis employs an automated test-case reduction algorithm called **shrinking**. When a failure is detected, Hypothesis simplifies the input to find the smallest, most concise counterexample that reproduces the defect:

- **Minimal Counterexamples Discovered:**
  - clean_dish_name: s = None (minimal null reference).
  - parse_item_prices: s = "" and s = None (empty string and null inputs).
  - split_amount: s = "" (missing delimiter), s = "/" (empty numerical tokens), and s = None.
- **Why Manual Tests Miss Them:** Developers write unit tests with cognitive bias, anticipating typical workflow inputs ("happy paths") and standard edge cases (e.g. negative numbers or large values). Manual testing rarely tests non-standard Unicode, unintended delimiter counts, null pointers, or empty string splits across all parameters simultaneously. Fuzzing explores the complete input domain systematically, finding edge cases that humans rarely imagine.

Question 3: How was defensive programming applied to harden the functions, and what are the benefits of fuzz testing?

To harden the application against all invalid and malformed inputs, we refactored each function using defensive programming principles:

- **Type & Null Guards:** Checked isinstance(input, str) and bool(input) upfront, returning safe default values ("" for strings, [] for lists, and 0 for numerical totals) instead of allowing unchecked execution.
- **Safe Token Parsing & Resilient Iteration:** In parse_item_prices, individual tokens are stripped and wrapped in a try-except (ValueError, TypeError) block, skipping malformed entries while retaining valid integers.
- **Delimiter Count & Arithmetic Safety:** In split_amount, verified len(parts) == 2 before unpacking, and explicitly checked if people == 0 alongside catching ZeroDivisionError and ValueError.
- **Benefits of Property-Based Fuzz Testing:** Fuzz testing ensures high reliability and security by discovering crash vulnerabilities, denial-of-service conditions, and injection surfaces before code is deployed. Combining Hypothesis property tests with defensive programming guarantees software resilience across the entire possible input space.